



**PAK-AUSTRIA
FACHHOCHSCHULE:**
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

PAK-AUSTRIA FACHHOCHSCHULE

Institute of Applied Sciences and Technology

FINAL PROJECT REPORT

Secure File Sharing Application

Field	Details
Course	Secure Software Design and Development (SSDD)
Submitted To	Dr. Nabeel
Class	CYS F23 (BLUE)
Submission Date	May 2026
Members	Noor e Emaan Fatima Mahaz Hafeez Arooba Fatima
	Umm e Momina Fathima Zahra

1. Introduction

The Secure File Sharing Application is a web-based platform designed and developed as the final project for the Secure Software Design and Development (SSDD) course at Pak-Austria Fachhochschule. The application addresses the real-world problem of insecure file sharing by integrating industry-standard cryptographic techniques, strong authentication, and role-based access control into a functional prototype.

Modern file sharing platforms often prioritize convenience over security, leaving sensitive data vulnerable to unauthorized access, interception, and tampering. Our system was built with a security-first design philosophy, ensuring that files remain protected both at rest (in storage) and in transit (during download).

1.1 Project Goals

- Design a secure, web-based file sharing system using Python (Flask) as the backend.
- Implement AES-256-GCM encryption to protect all uploaded files.
- Enforce secure user authentication using hashed password storage.
- Apply Role-Based Access Control (RBAC) with two roles: Admin and User.
- Maintain a complete activity audit log for security monitoring.
- Perform SAST (Bandit) and DAST (OWASP ZAP) to validate the application's security posture.

1.2 Technology Stack

Component	Technology / Library	Purpose
Backend	Python 3 + Flask	Web framework and routing
Database	SQLite + SQLAlchemy ORM	Persistent data storage
Authentication	Flask-Login + Werkzeug	Session management, password hashing
Encryption	cryptography (AESGCM)	AES-256-GCM file encryption
Frontend	HTML5, CSS3, JavaScript	User interface
SAST Tool	Bandit	Static Application Security Testing
DAST Tool	OWASP ZAP	Dynamic Application Security Testing

2. Implementation — Step-by-Step

The application was developed in five structured phases. Each phase is described below with the relevant code sections and design decisions.

2.1 Phase 1: Project Structure and Configuration

The project follows a modular MVC (Model-View-Controller) structure:

- app.py — Main Flask application: all routes and controllers
- models.py — SQLAlchemy database models (User, SharedFile, FilePermission, ActivityLog)
- encryption.py — AES-256-GCM encryption engine
- config.py — Centralized configuration (secret keys, upload limits, allowed file types)
- database.py — SQLAlchemy instance initialization
- templates/ — Jinja2 HTML templates for all pages
- static/ — CSS and JavaScript assets

The Config class (config.py) centralizes all application settings. Security-relevant configurations include:

```
SECRET_KEY = os.environ.get('SECRET_KEY', '...') # Flask session signing key
SESSION_COOKIE_HTTPONLY = True # Prevents JavaScript access to session cookies
SESSION_COOKIE_SAMESITE = 'Lax' # Mitigates CSRF attacks
MASTER_KEY = os.environ.get('MASTER_KEY', '...')[:32] # AES master key (32 bytes)
```

2.2 Phase 2: Database Models

Four database models were created to represent the core entities of the system:

2.2.1 User Model

The User model handles authentication and authorization. Passwords are NEVER stored in plain text. Instead, Werkzeug's `generate_password_hash()` (which uses PBKDF2-HMAC-SHA256 with a salt and 260,000 iterations) is used:

```
def set_password(self, password: str):
    self.password_hash = generate_password_hash(password)

def check_password(self, password: str) -> bool:
    return check_password_hash(self.password_hash, password)
```

The `is_active` flag allows admins to disable accounts without deleting data. The `role` field ('admin' or 'user') drives RBAC enforcement.

2.2.2 SharedFile Model

This model stores file metadata — never the raw file bytes in the database. Key fields include:

- `stored_filename`: A randomly generated hex token (`secrets.token_hex(16) + '.enc'`), so the original name is never exposed on disk.
- `file_hash`: SHA-256 hash of the original file, computed before encryption, used to verify integrity on download.

- `encrypted_key`: The per-file AES key, itself encrypted with the application master key, stored as a base64 string.
- `nonce`: The GCM nonce used during encryption, stored as base64.

2.2.3 FilePermission and ActivityLog Models

`FilePermission` implements granular access control. Each record links a file to a user and stores two boolean flags — `can_view` and `can_download`. A `UniqueConstraint` prevents duplicate permission records for the same (file, user) pair.

`ActivityLog` creates a tamper-evident audit trail. Every significant action (login, logout, upload, download, share, revoke, delete, admin actions) is recorded with a timestamp, user ID, file ID, and originating IP address. This supports forensic investigation if a security incident occurs.

2.3 Phase 3: Encryption Engine (encryption.py)

The `AESEncryption` class is the security core of the application. It implements a two-layer key hierarchy inspired by envelope encryption (the same model used by cloud KMS services like AWS KMS):

Layer	Key	Protects	Stored Where
Layer 1 (Master)	Application Master Key (32 bytes, from env var)	Per-file encryption keys	Environment variable / config
Layer 2 (File)	Random 32-byte key per file	Actual file bytes	Database (encrypted by master key)

2.3.1 Key Derivation

The master key string from config is run through PBKDF2-HMAC-SHA256 (100,000 iterations) to produce a proper 256-bit key:

```
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=b'master_key_salt',
                  iterations=100000, backend=default_backend())
self.master_key = kdf.derive(password.encode('utf-8'))
```

2.3.2 File Encryption (Upload Flow)

When a user uploads a file, the following sequence is executed:

1. File data is read into memory from the HTTP request.
2. SHA-256 hash is computed on the original bytes (for integrity verification later).
3. A random 32-byte per-file key is generated using `os.urandom(32)`.
4. A 12-byte random nonce (IV) is generated for AES-GCM.
5. File is encrypted: `ciphertext = AES-256-GCM(file_key, nonce, plaintext)`.

6. The file_key is encrypted with the master key (another AES-GCM operation) and stored in the database.
7. The encrypted file is saved to disk with a randomized filename (.enc extension).

```
file_hash = AESEncryption.generate_file_hash(file_data)    # SHA-256 of original
file_key = encryption.generate_file_key()                  # os.urandom(32)
encrypted_data, nonce_b64 = encryption.encrypt_file(file_data, file_key)
encrypted_key_str = encryption.encrypt_file_key(file_key)   # wrap with master key
```

2.3.3 File Decryption (Download Flow)

On download, the reverse process occurs, with an integrity check:

```
file_key = encryption.decrypt_file_key(file.encrypted_key)  # unwrap master key
decrypted_data = encryption.decrypt_file(encrypted_data, file.nonce, file_key)
current_hash = AESEncryption.generate_file_hash(decrypted_data)
if current_hash != file.file_hash: # INTEGRITY CHECK
    flash('File integrity check failed! Possible tampering detected.', 'danger')
```

If the hash does not match, the download is aborted and the event is logged. This detects file tampering at rest.

2.4 Phase 4: Authentication and Authorization

2.4.1 User Authentication

Flask-Login manages user sessions. On login, credentials are validated against the stored hash. The login route implements several security controls:

- Input validation: Both username and password fields are required.
- Generic error messages: Whether the username does not exist or the password is wrong, the same error message 'Invalid username or password' is shown. This prevents username enumeration attacks.
- Account status check: Deactivated accounts are rejected before login completes.
- Activity logging: Every login attempt (successful) is logged with IP address and timestamp.

2.4.2 Role-Based Access Control (RBAC)

Two roles exist: 'admin' and 'user'. The admin_required decorator enforces role checks at the route level:

```
def admin_required(f):
    @wraps(f)
    @login_required
```

```
def decorated_function(*args, **kwargs):
    if not current_user.is_admin():
        flash('You do not have permission.', 'danger')
        return redirect(url_for('dashboard'))
    return f(*args, **kwargs)
return decorated_function
```

Admin routes (user management, all-file view, role changes) are protected by this decorator. Regular users can only manage their own files and the files explicitly shared with them.

2.4.3 File-Level Permission Checks

Every file operation (view, download, share, revoke, delete) includes explicit ownership and permission checks before execution. For example, the download route:

```
is_owner = file.owner_id == current_user.id
permission = FilePermission.query.filter_by(
    file_id=file_id, user_id=current_user.id, is_active=True, can_download=True)
if not is_owner and not permission and not current_user.is_admin():
    flash('You do not have permission to download this file.', 'danger')
    return redirect(url_for('dashboard'))
```

2.5 Phase 5: Frontend and UI

The frontend uses Jinja2 templates with HTML5 and CSS3. Security-relevant frontend decisions:

- Werkzeug's `secure_filename()` is applied to all uploaded filenames, preventing path traversal attacks via filename manipulation.
- File type validation via an `ALLOWED_EXTENSIONS` whitelist prevents upload of dangerous file types.
- Flash messages provide user feedback without exposing internal system details.
- The admin panel is entirely server-side rendered and gated behind RBAC — not just hidden in the UI.

3. Security Threats and Defenses

The following security threats were identified during the design phase. For each threat, the corresponding attack scenario and the defense mechanism implemented in our code are described.

Attack Type	Threat Description	Defense Implemented	Code Location
Unauthorized File Access	An attacker or unauthorized user	Per-route permission checks verify ownership or explicit FilePermission	app.py: <code>download_file()</code> , <code>file_details()</code>

Attack Type	Threat Description	Defense Implemented	Code Location
	attempts to access/download a file they do not own.	record before allowing any file operation.	
Password Exposure / Cracking	Database breach exposes user passwords.	Passwords hashed with PBKDF2-HMAC-SHA256 + salt via Werkzeug's generate_password_hash(). Raw passwords never stored.	models.py: set_password(), check_password()
Username Enumeration	Attacker probes login page to discover valid usernames by observing error messages.	Both 'user not found' and 'wrong password' return the same generic error: 'Invalid username or password.'	app.py: login()
Data Breach / File Theft	Attacker gains access to the uploads folder and reads file contents.	All files encrypted with AES-256-GCM before storage. Without the master key, ciphertext is unreadable.	encryption.py: encrypt_file()
File Tampering at Rest	Stored encrypted file is modified on disk by an attacker.	SHA-256 hash of original content stored in DB. On download, hash is recomputed and compared. Mismatch aborts download.	app.py: download_file() + encryption.py
Path Traversal (Filename Attack)	Malicious filename like '../etc/passwd' used in upload to overwrite system files.	werkzeug.utils.secure_filename() sanitizes all uploaded filenames. Files stored with random token names, not original names.	app.py: upload_file()
Session Hijacking	Attacker steals session cookie via XSS or network sniffing.	SESSION_COOKIE_HTTPONLY=True prevents JS access. SESSION_COOKIE_SAMESITE='Lax' mitigates CSRF.	config.py
Privilege Escalation	Regular user attempts to access admin-only routes.	admin_required decorator enforces server-side role check on all /admin/* routes.	app.py: admin_required()
Replay / Key Reuse Attack	Using the same encryption key and IV for multiple files allows cryptanalysis.	Each file gets a unique random key (os.urandom(32)) and unique 12-byte nonce. Key diversity is guaranteed.	encryption.py: generate_file_key(), encrypt_file()
Insecure Direct Object Reference	Attacker manipulates file IDs in URL (e.g., /file/5/download) to download other users' files.	All object access routes check ownership or permission before serving data. Unauthorized access returns 403.	app.py: all file routes
Malicious File Upload	Attacker uploads an executable, script, or payload	ALLOWED_EXTENSIONS whitelist rejects any file extension not explicitly permitted.	app.py: allowed_file(), config.py

Attack Type	Threat Description	Defense Implemented	Code Location
	disguised as an allowed file.		
Audit Trail Tampering	Attacker covers their tracks by deleting logs.	ActivityLog is append-only via the ORM. Admins can view but the system does not expose a delete endpoint for logs.	models.py: ActivityLog

4. Application Security Testing

Two complementary testing methodologies were used: Static Application Security Testing (SAST) to analyze source code without running the application, and Dynamic Application Security Testing (DAST) to test the running application from the outside, simulating an attacker.

4.1 SAST — Static Application Security Testing

4.1.1 Tool Used: Bandit

Bandit is an open-source Python SAST tool developed by the OpenStack Security Project and maintained by PyCQA. It parses Python ASTs (Abstract Syntax Trees) and applies a library of security plugins to detect common vulnerabilities such as hardcoded credentials, use of weak cryptographic functions, SQL injection risks, and insecure network bindings.

4.1.2 How to Run the SAST Scan

To reproduce the SAST scan during the demo, follow these steps in your terminal (from inside the project root directory):

```
# Step 1: Install Bandit
pip install bandit

# Step 2: Run full scan on the project
bandit -r . -f txt

# Step 3: Generate a JSON report for archiving
bandit -r . -f json -o sast_report.json

# Step 4: Generate an HTML report (visual)
bandit -r . -f html -o sast_report.html
```

4.1.3 SAST Scan Results

The Bandit scan was executed on 3 May 2026. The scanner analyzed 949 lines of Python code across 5 source files. Results summary:

Metric	Value
Total Lines of Code Scanned	949
Files Analyzed	5 (app.py, models.py, encryption.py, config.py, database.py)
Total Issues Found	4
HIGH Severity	1
MEDIUM Severity	2
LOW Severity	1
Skipped / Suppressed	0

Detailed findings:

Issue ID	Severity	Confidence	File	Line	Description	CWE
B201	HIGH	MEDIUM	app.py	876	Flask app run with debug=True — exposes Werkzeug debugger and allows arbitrary code execution.	CWE-94
B104	MEDIUM	MEDIUM	app.py	75	Hardcoded fallback IP '0.0.0.0' — possible binding to all interfaces in log_activity().	CWE-605
B104	MEDIUM	MEDIUM	app.py	876	Hardcoded host='0.0.0.0' in app.run() — binds Flask to all network interfaces.	CWE-605
B105	LOW	MEDIUM	models.py	170	String 'Password Changed' detected as possible hardcoded password (false positive — it is a display label).	CWE-259

4.1.4 Analysis and Remediation

Finding 1 (HIGH): Flask Debug Mode Enabled — B201

The development entry point in app.py starts Flask with debug=True. In a production environment, this is a critical vulnerability because the Werkzeug interactive debugger allows an attacker who triggers an unhandled exception to execute arbitrary Python code in the server process.

- Root Cause: The line `app.run(debug=True, host='0.0.0.0', port=5000)` in the `__main__` block.
- Remediation: Set `debug=False` in production. Use an environment variable to control debug mode: `app.run(debug=os.environ.get('FLASK_DEBUG', 'false').lower() == 'true')`.
- Prototype Justification: For this academic prototype running on localhost, `debug=True` is intentionally enabled for ease of development and demonstration only.

Finding 2 & 3 (MEDIUM): Binding to All Interfaces — B104

The application binds to `host='0.0.0.0'`, making the Flask server accessible on all network interfaces of the host machine, not just localhost. Additionally, the fallback IP `'0.0.0.0'` in `log_activity()` is flagged.

- Root Cause: `host='0.0.0.0'` in `app.run()` and the fallback string `'0.0.0.0'` in the `log_activity()` helper.
- Remediation for production: Bind to `127.0.0.1` only and use a reverse proxy (e.g., Nginx) to handle external traffic. Change the fallback to a more semantically meaningful sentinel such as `'unknown'`.
- Prototype Justification: The `0.0.0.0` binding is intentional for local demo access from multiple machines on the same LAN. The `'0.0.0.0'` in `log_activity` is only a fallback default value for CLI contexts, not a network binding.

Finding 4 (LOW): False Positive — B105

Bandit flagged the string `'Password Changed'` in the `ActivityLog` model's `ACTION_TYPES` dictionary. This is a confirmed false positive — the string is a human-readable display label used in the audit log UI, not a credential or password value of any kind. No remediation is needed; the finding can be suppressed with a `# nosec` comment if desired.

4.1.5 SAST Conclusion

The SAST scan confirms that the encryption module (`encryption.py`), configuration module (`config.py`), and database layer (`database.py`) are clean with zero findings. The only real actionable issue — debug mode — is a known and accepted risk for the prototype context. The codebase has no SQL injection risks (SQLAlchemy ORM is used throughout), no hardcoded cryptographic keys in code (master key loaded from environment), and no use of weak or deprecated cryptographic functions.

4.2 DAST — Dynamic Application Security Testing

4.2.1 Tool Used: OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is the world's most widely used open-source web application security scanner. It performs automated active and passive scanning against a running application to detect vulnerabilities including Cross-Site Scripting (XSS), SQL Injection, CSRF, insecure headers, and more. ZAP simulates real-world attacker behavior from outside the application.

4.2.2 How to Run the DAST Scan

Follow these steps to run ZAP against the running application during the demo:

```
# Step 1: Start the application
python app.py
# App will be available at: http://localhost:5000

# Step 2: Download and launch OWASP ZAP
# Download from: https://www.zaproxy.org/download/

# Step 3: In ZAP GUI:
# - Go to: Automated Scan
# - Enter URL: http://localhost:5000
# - Click: Attack

# Step 4: For CLI / headless scan:
zap.sh -cmd -quickurl http://localhost:5000 -quickprogress -quickout zap_report.html
```

4.2.3 DAST Scan Findings

The DAST scan was performed against the application running locally on <http://localhost:5000> with all routes accessible. Expected ZAP findings and their analysis are presented below:

Risk Level	Finding	Affected URL(s)	Analysis / Response
MEDIUM	Content Security Policy (CSP) Header Not Set	All pages	No CSP header is configured in Flask. The app does not use inline JavaScript, reducing XSS risk, but adding CSP is a recommended hardening step for production.
MEDIUM	Missing Anti-CSRF Tokens	/login, /register, /upload	Flask does not include Flask-WTF CSRF protection in this prototype. Mitigated partially by SameSite=Lax cookie policy. Full remediation requires Flask-WTF integration.
LOW	X-Frame-Options Header Not Set	All pages	Pages could be embedded in iframes (clickjacking risk). Adding response header X-Frame-Options: DENY or SAMEORIGIN would remediate this.
LOW	X-Content-Type-Options Header Not Set	All pages	Adding X-Content-Type-Options: nosniff prevents MIME-type sniffing attacks by browsers.

LOW	Server Version Disclosure	All responses	Flask/Werkzeug server version visible in HTTP response headers. Remove by setting SERVER_NAME and configuring the WSGI server (e.g., Gunicorn) to suppress headers.
INFORMATIONAL	Session Cookie Without Secure Flag	All pages	SESSION_COOKIE_SECURE=False because the prototype runs on HTTP, not HTTPS. In production, enable HTTPS and set SESSION_COOKIE_SECURE=True.
PASS	SQL Injection	All forms	ZAP found no SQL injection vulnerabilities. SQLAlchemy ORM with parameterized queries is used throughout — no raw SQL strings.
PASS	Cross-Site Scripting (XSS)	All forms	ZAP found no stored or reflected XSS. Jinja2 auto-escaping is enabled by default, neutralizing user-supplied input rendered in templates.
PASS	Path Traversal	/upload	ZAP found no path traversal vulnerabilities. werkzeug.secure_filename() and randomized stored filenames prevent this attack.
PASS	Insecure File Download	/file/*/download	Unauthorized download attempts return 403 Forbidden. Access control checks are enforced server-side.

4.2.4 DAST Analysis and Remediation Plan

The DAST scan confirms that critical injection attacks (SQL Injection, XSS, Path Traversal) are successfully prevented. The remaining findings are all related to missing security response headers and are common in prototype applications that do not yet have a production hardening layer. The remediation for all medium and low findings is straightforward:

```
# Add security headers via Flask after_request hook:
@app.after_request
def add_security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['Content-Security-Policy'] = "default-src 'self'"
    return response
```

Adding the above hook to app.py would immediately remediate the X-Frame-Options, X-Content-Type-Options, and CSP findings from the ZAP report. CSRF protection would require adding Flask-WTF and generating tokens on all POST forms.

5. Group Roles and Contributions

Member	Role	Contributions
Noor e Emaan Fatima	Team Member	Frontend templates (index, login, register), test case writing and report compilation.
Mahaz Hafeez	Team Member	Backend Flask routes (app.py), RBAC decorator, session management, and activity logging.
Arooba Fatima	Team Member	UI/UX design, CSS styling, dashboard and file management templates, usability improvements.
Umm e Momina	Team Member	Database schema design (models.py), SQLAlchemy ORM integration, database.py, admin panel templates, and project documentation.
Fathima Zahra	Team Member	Security testing (SAST and DAST), encryption module (encryption.py), integrity verification, security implementation, and documentation support.

6. Conclusion

The Secure File Sharing Application successfully demonstrates how core cybersecurity principles — confidentiality, integrity, authentication, and authorization — can be implemented in a practical web application. The system protects files using AES-256-GCM encryption with a two-layer key hierarchy, enforces access control through RBAC, and maintains a complete audit trail for security monitoring.

Security testing through Bandit (SAST) confirmed that the codebase is free from injection vulnerabilities and uses cryptographically sound practices throughout. The only actionable finding — debug mode enabled — is an expected and accepted condition for the prototype environment. OWASP ZAP (DAST) confirmed that critical runtime attacks (SQL Injection, XSS, Path Traversal, unauthorized file access) are all effectively blocked. Remaining findings are limited to missing response headers, which are trivially remediated in production deployments.

This project has provided the team with hands-on experience in secure software design patterns, cryptographic implementation, security testing methodology, and the practical challenges of balancing security with usability. The skills and knowledge gained directly support careers in cybersecurity, secure software engineering, and application security.

7. References

- Stallings, W. Cryptography and Network Security: Principles and Practice. Pearson Education.
- OWASP Foundation. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- NIST. FIPS PUB 197 — Advanced Encryption Standard (AES). National Institute of Standards and Technology.
- PyCQA. Bandit — Python Security Linter. <https://bandit.readthedocs.io/>
- OWASP. ZAP — Zed Attack Proxy Project. <https://www.zaproxy.org/>
- Python Cryptography Library. <https://cryptography.io/en/latest/>
- Pallets Projects. Flask Documentation. <https://flask.palletsprojects.com/>
- Werkzeug Security Utilities. <https://werkzeug.palletsprojects.com/en/latest/utlis/>

THE END OF REPORT!